# CSC_51053_EP, Exercise Session 4: SQL queries with Generative AI

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

October 17, 2025

## 1 Overview

The purpose of this session is to experiment with the capabilities of trained Large Language Models (LLMs, in short) for generating SQL queries from a question expressed in natural language. Since SQL has been widely used for decades, numerous teaching and technical resources exist online, which have been used to train existing LLMs[1]. We will use those of OpenAI, via the paying API, thanks to a dedicated grant from Ecole polytechnique.

## 2 Setting up

For this lab, you will need **Python-3.12** installed on your computer. We provide a zip'ed directory containing a `readme` file.

> Follow the instructions in `readme` to create a python virtual environment and obtain SQL queries from OpenAI's API.

In this lab:

- The **schema** of the relational database that we work on will be in a **prompt file**, e.g., `gift-prompt`.
- The **natural-language specification of the query** will be in the `query` variable in the python script `prompt-llm.py`.
- You will receive an **API key** which will enable you to use the OpenAI API from your python script. Save this key in a file in the same directory.

## 3 Gift queries

We revisit the database introduced in the previous session (Figure 1).

> Using the `gpt4` model, replace, successively, the query in `gift-prompt` with each of the four queries in `gift-query-list.txt` file (there is one query per line). Save and analyze each query you obtained. Are they correct?

The *temperature* of a model is a parameter that controls how much randomness (variability) is encouraged in the output, as opposed to "quasi deterministic" (temperature of 0). With a temperature of 0, the model returns most likely answers (these may still vary); with higher temperature values, more variable answers are returned.

> Replace the `temperature` value set to 0 in the code, with 1.2. Try again some queries. What do you notice about the queries? What about the comments surrounding them?

[1] While public knowledge about the training procedure is limited, it is known that they have been trained on large collections of technical books, e.g., shows that all versions of Raghu Ramakrishnan's textbooks are in the Library Genesis (LibGen) dataset on which all major LLMs are trained.

Gift database description: pooling money to buy a gift

create table **guser**(uid int primary key, uname varchar); – *people who pool money to buy a gift*
create table **category**(cid int primary key, cname varchar);
– *e.g., 'sport', 'art', 'cooking' etc.*
create table **gift**(id int primary key, descr varchar, price int, cID int references category(cid));
– *gifts that could be bought*
create table **contribution**(uid int primary key references guser(uid), amount int not null);
– *each user may contribute an amount of money to the gift*
create table **preference**(uid int references guser(uid), cid int references category(cid), porder int not null check (1<=porder and porder<=3), primary key (uid, cid, porder));
– *each user may specify at most 3 preferences for the category of the gift to be bought. A user preference order (porder) of 1 is the option the user favors most; 2 is the "next" preference order, and 3 is the least favored preference order*

If a user $u$ put a category $c$ first in her preference order (porder=1), we view this as a *support score* of 3; similarly, porder=2 means a support score of 2, while porder=3 means a support of 1.

Figure 1: Recall of the gift database schema.

Modify the input to ask the model to only return the queries, not comments or explanations. (The monetary cost paid to OpenAI is per-token sent to or received from the model.)

Now change the model to `gpt5`, *remove the temperature parameter* which we cannot set with this model (the default is 1.0) and try again the same queries. Save and analyze each query. Are they correct?

You may also test the queries in PostgreSQL, on the sample database from the previous lab.

## 4 Nonsensical database

We now consider a simple database consisting of three tables, where table and attribute names do not make any sense. They are described in `nonsensedb-prompt`.

Prompt `gpt4` and `gpt5` with each natural language query in `nonsensedb-query-list.txt` (again, there is one per line). Analyze the result.

Edit `nonsensedb-prompt` to keep just the first table (delete the two others). Prompt again an LLM with the questions. What happens? What can we conclude?

## 5 Large schema (US census database)

Unlike our exercise databases, real-life ones may have very large schemas.

On the US census dataset characterized by the schema in `census-prompt` (278 tables), try to obtain a query for the question in `census-query-list.txt` from `gpt5`. What happens?

Do the same thing but using the reduced prompt `census-prompt-small` (9 tables). What happens? Now try the same on `gpt4`.